

Relatório de Análise — Algoritmos de Busca em Strings

Explicação dos algoritmos implementados, análise comparativa de desempenho e discussão sobre quando cada algoritmo é mais eficiente, com comparação entre o comportamento teórico e o prático medido em arquivos grandes e reais.

1. Algoritmos implementados

1.1 Naive (força bruta)

Para cada posição possível do texto, tenta casar o padrão caractere a caractere, da esquerda para a direita. Não usa pré-processamento. Simples, mas pode repetir muitas comparações. **Complexidade: $O(n \cdot m)$** no pior caso.

1.2 Rabin-Karp

Usa um **hash de rolagem** (rolling hash): calcula a impressão digital do padrão e de cada janela do texto em $O(1)$ por deslocamento. Só faz a comparação caractere a caractere quando os hashes coincidem (para descartar colisões). **Complexidade: $O(n+m)$ no caso médio**; $O(n \cdot m)$ no pior caso.

1.3 Knuth-Morris-Pratt (KMP)

Pré-processa o padrão construindo a tabela **LPS** (Longest Proper Prefix which is also Suffix). Ao falhar uma comparação, em vez de voltar no texto, usa a LPS para reaproveitar o prefixo já casado. O texto nunca é re-lido. **Complexidade: $O(n+m)$** garantida.

1.4 Boyer-Moore

Compara o padrão da **direita para a esquerda** e, ao falhar, usa a tabela **bad-character** (último índice de cada caractere no padrão) para deslocar o padrão o máximo possível, frequentemente pulando vários caracteres de uma vez. **Complexidade: $O(n/m)$ no melhor caso**; $O(n \cdot m)$ no pior caso.

2. Metodologia

- **Textos:** gerados com palavras pseudo-naturais, com o padrão "algoritmo" injetado periodicamente para garantir ocorrências.
- **Tamanhos:** ~64 KB, ~1 MB e ~5 MB. **Padrão:** "algoritmo" ($m = 9$).
- **Medição:** média de 20 execuções por algoritmo (tempo com `performance.now()`).
- **Métricas:** comparações de caractere (independente de hardware) e tempo (ms).
- **Instrumentação:** cada execução passa pelo SearchContext, que gera traces, métricas e logs via OpenTelemetry.

3. Resultados medidos

Número de comparações de caractere

Texto	Naive	Rabin-Karp	KMP	Boyer-Moore
~64 KB	77.765	468	75.880	11.692
~1 MB	1.244.364	7.488	1.214.074	186.509
~5 MB	6.221.830	37.431	6.070.356	932.628

Tabela 1 — Comparações de caractere por algoritmo e tamanho de texto.

Tempo médio de execução (ms)

Texto	Naive	Rabin-Karp	KMP	Boyer-Moore
~64 KB	0,448	1,213	0,456	0,359
~1 MB	3,496	16,700	4,196	2,049
~5 MB	16,816	83,657	21,084	10,687

Tabela 2 — Tempo médio (ms). Ocorrências idênticas para os quatro: 52, 832 e 4.159.

4. Teórico × prático — interpretação

Crescimento linear com n . Para todos os algoritmos, ao multiplicar o texto por ~ 5 (1 MB $\rightarrow$ 5 MB), tanto comparações quanto tempo crescem $\sim 5\times$, coerente com o termo dominante proporcional a n para um padrão fixo.

Naive e KMP ficam próximos neste cenário. Ambos têm número de comparações parecido ($\sim 6,2M$ vs $\sim 6,1M$ em 5 MB). O padrão "algoritmo" tem baixa sobreposição interna (LPS quase toda zero) e o texto raramente forma prefixos parciais longos; logo o KMP quase não tem o que pular. A vantagem do KMP aparece em padrões com muita repetição (ex.: "aaaab" em "aaaaaaaa..."), onde o Naive degrada para $O(n \cdot m)$.

Boyer-Moore é o mais rápido e faz menos comparações (entre os que comparam caractere): ~ 933 mil em 5 MB, $\sim 6,5\times$ menos que o Naive, porque a heurística bad-character pula trechos inteiros. É o vencedor prático em alfabetos grandes e padrões médios/longos — o caso típico de busca em texto natural.

Rabin-Karp tem pouquíssimas comparações de caractere, mas o maior tempo. Faz apenas ~ 37 mil comparações em 5 MB (só nos acertos de hash), porém é o mais lento no relógio (~ 84 ms). O trabalho real está no cálculo do hash de rolagem (aritmética modular por caractere), que o $O(n+m)$ teórico abstrai: a métrica de comparações sozinha subestima seu custo; só o tempo revela a constante oculta.

5. Quando cada algoritmo é mais eficiente

- **Naive** — textos/padrões pequenos ou quando a simplicidade importa mais que o desempenho. Bom baseline didático.
- **Rabin-Karp** — busca de múltiplos padrões simultânea ou detecção de plágio/duplicatas, onde o hash é reaproveitado. Em padrão único costuma perder pelo custo do hash.
- **KMP** — quando há garantia $O(n+m)$ necessária e o padrão tem muita repetição interna (alfabetos pequenos, padrões periódicos). Streaming de texto.
- **Boyer-Moore** — melhor escolha geral para busca em texto natural com alfabeto grande e padrões médios/longos; quanto maior o padrão, maiores os saltos.

6. Observações de engenharia

Comparações vs. tempo de relógio. Comparar caracteres é a métrica mais fiel à teoria por não depender de hardware/JIT/cache; mas, como mostra o Rabin-Karp, ela não captura todo o custo — por isso medimos as duas coisas.

Papel da observabilidade. A teoria prevê a tendência; a prática revela constantes ocultas e efeitos de plataforma. A instrumentação OpenTelemetry + dashboard tornou possível coletar e comparar essas evidências de forma sistemática.

7. Conclusão

Os resultados empíricos confirmam as classes de complexidade teóricas e, ao mesmo tempo, expõem nuances que a notação assintótica esconde: a vantagem do KMP depende da estrutura do padrão, e o Rabin-Karp paga caro pelo hash apesar de poucas comparações. Para busca de padrão único em texto natural, **Boyer-Moore** foi o mais eficiente em todos os tamanhos testados.